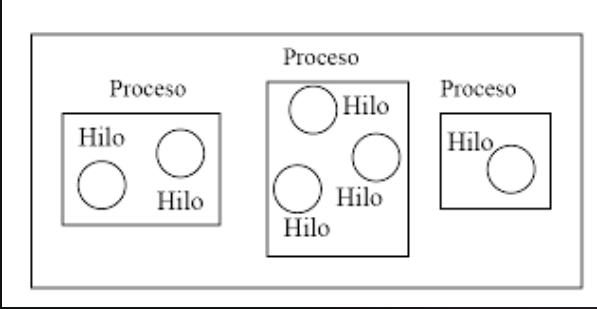


SINCRONIZACIÓN ENTRE PROCESOS/HILOS



INTRODUCCIÓN - MONITORES

Los semáforos son una herramienta eficaz para coordinar procesos en sistemas concurrentes. No obstante, su uso inadecuado puede causar errores de sincronización complejos de identificar. Estos fallos solo se presentan en circunstancias específicas de ejecución, lo que complica su detección.

Problemas con semáforos

Problema del productor-consumidor: Este caso ilustra cómo el uso de contadores en una solución puede producir errores de temporización. Aunque los valores puedan parecer correctos, pueden diferir del valor real y provocar problemas de sincronización.

Problemas con semáforos

Errores de temporización incluso con semáforos: Aunque se empleen semáforos para resolver problemas de sincronización, pueden seguir ocurriendo errores de temporización. Se muestra cómo estos errores pueden aparecer incluso cuando se utilizan semáforos.

Errores comunes

Alteración en el orden de ejecución de operaciones: Se explican casos en los que modificar el orden de las operaciones `wait()` y `signal()` puede ocasionar una violación de la exclusión mutua.

Errores comunes

Reemplazo incorrecto de operaciones:

Se explica cómo un reemplazo inadecuado de operaciones, como sustituir `signal()` por `wait()`, puede causar interbloqueos.

Errores comunes

Omisión de operaciones críticas: Se describe cómo la falta de operaciones `wait()`, `signal()`, o ambas, puede provocar violaciones de la exclusión mutua o dar lugar a interbloqueos.

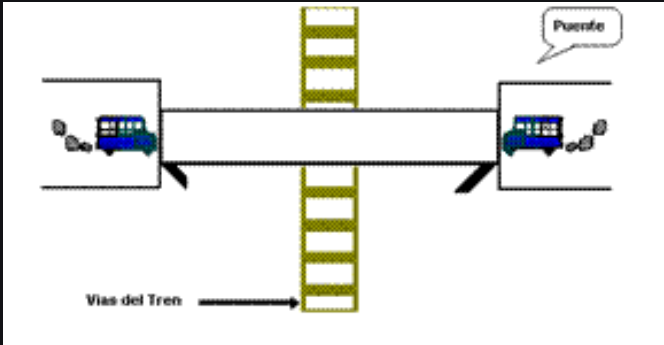
CONSECUENCIAS DE ERRORES

Interbloqueo:

Se explica cómo los errores de sincronización pueden generar situaciones en las que los procesos quedan bloqueados de manera indefinida, incapaces de avanzar debido a conflictos en el acceso a recursos compartidos.

CONSECUENCIAS DE ERRORES

Violación de exclusión mutua: Se explica cómo los errores de sincronización pueden permitir que múltiples procesos accedan a secciones críticas simultáneamente, lo que viola el requisito de exclusión mutua.



SOLUCIÓN: MONITORES

Estructura de sincronización de alto nivel
Se introduce el concepto de monitores como una solución de alto nivel para resolver los errores comunes de sincronización.

SOLUCIÓN: MONITORES

Aborda errores comunes de sincronización
Se destaca cómo los monitores proporcionan una interfaz más segura y fácil de utilizar, lo que disminuye la probabilidad de cometer errores de sincronización.

SOLUCIÓN: MONITORES

Proporciona un marco más seguro y fácil de usar: Se resalta la ventaja de los monitores sobre los semáforos en términos de facilidad de uso y prevención de errores de sincronización.

INTRODUCCIÓN - UTILIZACIÓN

Un monitor es un tipo abstracto de datos que agrupa datos privados junto con métodos públicos. Los métodos públicos operan sobre esos datos y garantizan exclusión mutua dentro del monitor. Un monitor incluye la definición de variables que representan el estado de una instancia, así como los procedimientos o funciones que interactúan con estas variables.

RESTRICCIONES DE ACCESO

- Los procedimientos dentro de un monitor solo pueden acceder a las variables locales del monitor y a sus parámetros formales.
- Las variables locales de un monitor solo pueden ser accedidas por los procedimientos locales.

EXCLUSIÓN MUTUA

- La estructura del monitor garantiza que solo un proceso esté activo dentro del monitor en un momento dado.
- Esta restricción de sincronización se implementa implícitamente, lo que elimina la necesidad de que el programador lo codifique explícitamente.

LIMITACIONES DE LA ESTRUCTURA DE MONITOR

- La estructura de monitor puede no ser suficientemente potente para algunos esquemas de sincronización.
- Se necesitan mecanismos de sincronización adicionales, proporcionados por la estructura condition.



UTILIZACIÓN DE VARIABLES DE CONDICIÓN

- Se pueden definir variables de tipo condition para esquemas de sincronización personalizados.
- Las únicas operaciones que se pueden realizar en una variable de condición son wait() y signal().
- La operación wait() suspende el proceso hasta que otro proceso invoque signal() en la misma variable de condición.

MANEJO DE PROCESOS SUSPENDIDOS

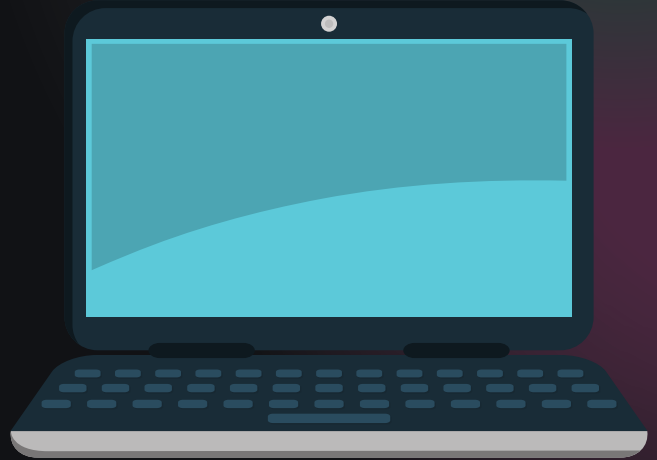
- Cuando un proceso invoca signal() en una variable de condición donde otro proceso está suspendido, existen dos posibilidades para continuar la ejecución.
- Se pueden adoptar diferentes enfoques, como señalar y esperar o señalar y continuar, dependiendo de la lógica del programa y los requisitos de sincronización.

COMPROMISO EN PASCAL CONCURRENTE

- En Pascal Concurrente, cuando un proceso ejecuta signal(), sale inmediatamente del monitor.
- Esta decisión permite que la ejecución del otro proceso se reanude de manera inmediata, comprometiendo entre las opciones de señalar y esperar o señalar y continuar.

SOLUCIÓN AL PROBLEMA DE LA CENA DE LOS FILÓSOFOS

El problema de la cena de los filósofos es un clásico en la ciencia de la computación y la teoría de la concurrencia. Propuesto por Edsger Dijkstra en 1965, ejemplifica los desafíos de la sincronización de procesos y el acceso a recursos compartidos.



DESCRIPCIÓN DEL PROBLEMA

Cinco filósofos están sentados alrededor de una mesa circular. Entre cada par de filósofos hay un tenedor. Para comer, cada filósofo necesita los dos tenedores que están a su lado. El problema ocurre cuando todos los filósofos intentan comer simultáneamente, lo que puede llevar a bloqueos mutuos.

SOLUCIÓN CLÁSICA

La solución clásica consiste en asignar un orden a los tenedores y exigir que los filósofos los tomen siguiendo ese orden para evitar bloqueos. Sin embargo, esta solución puede generar problemas de inanición si un filósofo no puede obtener su tenedor debido a la ocupación continua de los demás.

SOLUCIÓN CON SEMÁFOROS

Una solución más avanzada emplea semáforos para gestionar el acceso a los tenedores. Cada tenedor se representa con un semáforo, y los filósofos deben adquirir ambos semáforos al mismo tiempo para poder comer. Esto previene tanto los bloqueos como la inanición, ya que los filósofos pueden esperar pacientemente si no logran obtener ambos tenedores simultáneamente.

IMPLEMENTACIÓN DE MONITORES CON SEMÁFOROS

- Un monitor se puede implementar utilizando semáforos para garantizar la exclusión mutua y la sincronización.
- Se utiliza un semáforo para controlar el acceso al monitor en su conjunto, asegurando que solo un hilo pueda ejecutar métodos del monitor a la vez.

PASOS PARA IMPLEMENTAR UN MONITOR CON SEMÁFOROS

- Crear un semáforo para controlar el acceso al monitor (semáforo del monitor).
- Utilizar semáforos adicionales para controlar el acceso a recursos compartidos dentro del monitor.

- Al iniciar un método del monitor, adquirir el semáforo del monitor para bloquear el acceso a otros hilos.
- Liberar el semáforo del monitor al finalizar el método para permitir que otros hilos accedan.

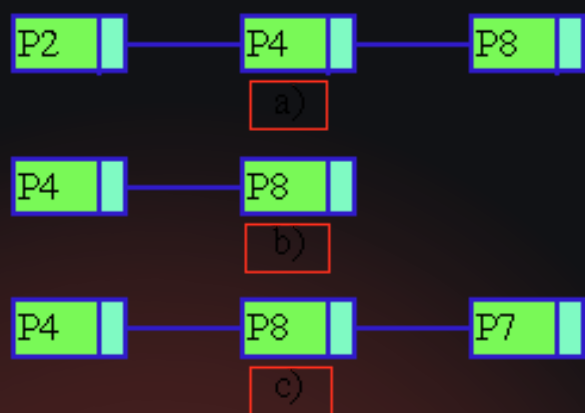


EJEMPLO DE IMPLEMENTACIÓN

- Ejemplo de un monitor que gestiona el acceso a una cola de mensajes:
- Métodos: `push_message`, `pop_message`.
- Semáforos: `sem_monitor`, `sem_queue` para controlar el acceso a la cola.

VENTAJAS

- Flexibilidad: Los semáforos permiten implementar diferentes políticas de sincronización según los requisitos.
- Eficiencia: Los semáforos son una primitiva de bajo nivel que puede adaptarse a diversas situaciones de concurrencia.



INTRODUCCIÓN - REANUDACIÓN DE PROCESOS DENTRO DE UN MONITOR

La reanudación de procesos en monitores es esencial para garantizar la sincronización adecuada en sistemas concurrentes. En este contexto, la correcta selección de procesos a reanudar después de una señalización es crítica. Exploraremos cómo se maneja este aspecto dentro de la estructura de un monitor, junto con sus implicaciones y desafíos.

PROBLEMA DE SELECCIÓN DE PROCESO

Al ejecutar `signal()` en una variable de condición con múltiples procesos suspendidos, surge la pregunta sobre cuál proceso continuación.

USO DE ORDEN DE PRIORIDAD

- Se puede usar un esquema de prioridad donde cada proceso tiene asociado un número de prioridad.
- Cuando se ejecuta `signal()`, se reanuda el proceso con el número de prioridad más bajo



EJEMPLO DE IMPLEMENTACIÓN

- Se muestra el monitor ResourceAllocator como un ejemplo que controla la asignación de un recurso entre procesos competidores.
- Los procesos solicitan acceso al recurso y especifican un tiempo máximo de uso.

SOLUCIÓN PROPUESTA

Una solución propuesta es incluir las operaciones de acceso al recurso dentro del monitor ResourceAllocator. Sin embargo, esto puede resultar en una planificación basada en el algoritmo del monitor en lugar del algoritmo codificado.

DESAFÍOS DE IMPLEMENTACIÓN

Se destacan desafíos como garantizar que los procesos realicen llamadas en la secuencia correcta y prevenir el acceso directo al recurso compartido sin utilizar los protocolos de acceso.

INSPECCIÓN Y CONTROL DE ACCESO

En sistemas pequeños y estáticos, la inspección puede garantizar la corrección del sistema. Sin embargo, en sistemas grandes o dinámicos, se necesitan mecanismos adicionales para el control de acceso.

INCORPORACIÓN EN LENGUAJES DE PROGRAMACIÓN

Varios lenguajes de programación han adoptado la idea de los monitores, entre ellos Pascal Concurrente, Mesa, C# y Java. Otros, como Erlang, proporcionan soporte de concurrencia similar.



FABRICA DE PROCESOS

CAPITULO 1 "PROCESOS"

"EN UNA FÁBRICA
DONDE LOS PROCESOS
DEBEN TRABAJAR
JUNTOS, SURGEN
PROBLEMAS CUANDO
NO SE COORDINAN
CORRECTAMENTE."

¡HOLA! SOY EL
PRODUCTOR. MI
TRABAJO ES GENERAR
ELEMENTOS PARA LA
FÁBRICA."



"Y YO SOY EL
CONSUMIDOR. CONSUMO
LOS ELEMENTOS QUE EL
PRODUCE, PERO A VECES
NOS DESINCRONIZAMOS."



SOY UNO DE LOS
FILÓSOFOS. ME
ENCANTA PENSAR...
PERO TAMBIÉN
NECESITO COMER, Y
ESOS PALILLOS SON
UN PROBLEMA."



SOY UN LECTOR.
LEO DATOS, PERO
NO PUEDO
HACERLO CUANDO
ALGUIEN ESTÁ
ESCRIBIENDO."



"¡SOY EL ESCRITOR!
NECESITO
ACTUALIZAR DATOS,
PERO LOS LECTORES
A VECES NO ME DEJAN
TRABAJAR."

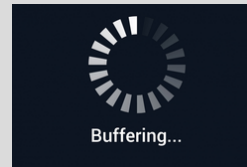


FABRICA DE PROCESOS

CAPITULO 2 "SEMAFOROS"

"EL PROBLEMA DEL
PRODUCTOR Y
CONSUMIDOR"

"¡NO PUEDO AÑADIR
MÁS ELEMENTOS, EL
BÚFER ESTÁ LLENO!"



"¡Y NO PUEDO CONSUMIR
PORQUE NO HAY NADA
DISPONIBLE!"



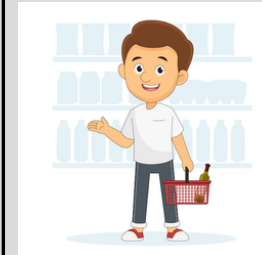
"USEN SEMÁFOROS.
PRODUCTOR, ESPERA
A QUE HAYA ESPACIO
ANTES DE PRODUCIR."



"¡ESPERARÉ
HASTA QUE HAYA
ESPACIO!"



"¡YO TAMBIÉN
ESPERARÉ HASTA QUE
HAYA ALGO PARA
CONSUMIR!"



"CON LOS
SEMÁFOROS,
EL PROBLEMA
SE RESOLVIÓ."



FABRICA DE PROCESOS

CAPITULO 3 ""

EL DILEMA DE LOS FILÓSOFOS

"¡TENGO HAMBRE, PERO NO PUEDO TOMAR DOS PALILLOS AL MISMO TIEMPO!"



"¡TAMPOCO YO! ESTO ES FRUSTRANTE."



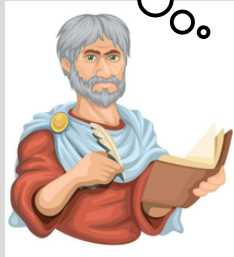
"PARA EVITAR INTERBLOQUEOS, USAREMOS UNA REGLA: SÓLO CUATRO FILÓSOFOS PUEDEN SENTARSE A LA MESA AL MISMO TIEMPO."



"¡FINALMENTE PUEDO COMER! ESTE SISTEMA FUNCIONA."



"¡GENIAL! AHORA PODEMOS PENSAR Y COMER SIN PROBLEMAS."



CAPITULO 4 "LOS LECTORES Y ESCRITORES"

"COMPARTIR RECURSOS Y ESCRIBIR"



"¡QUIERO LEER, PERO NO PUEDO PORQUE ESTÁS ESCRIBIENDO!"

"¡Y NO PUEDO ESCRIBIR PORQUE USTEDES QUIEREN LEER AL MISMO TIEMPO!"



FABRICA DE PROCESOS



"USAREMOS UN CERROJO LECTOR-ESCRITOR. LOS LECTORES PUEDEN ACCEDER AL RECURSO SIMULTÁNEAMENTE, PERO LOS ESCRITORES TENDRÁN EXCLUSIVIDAD."

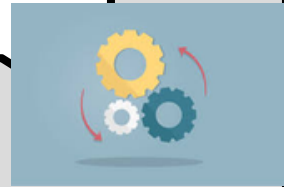


"¡AHORA PODEMOS LEER SIN INTERRUPIRNOS!"



"¡Y YO PUEDO ESCRIBIR CUANDO LO NECESITO!"

"GRACIAS AL TRABAJO EN EQUIPO Y AL USO DE SEMÁFOROS Y CERROJOS, TODOS LOS PROCESOS EN LA FÁBRICA FUNCIONAN EN ARMONÍA."



"RECUERDEN, LA SINCRONIZACIÓN ES LA CLAVE PARA EVITAR INTERBLOQUEOS E INANICIÓN."

FIN

"¡SINCRONIZACIÓN PARA UNA FÁBRICA EFICIENTE!"

SINCRONIZANDO PROBLEMAS DE IPC

CAPITULO 1 "BIENVENIDO A MI MUNDO"

"BIENVENIDOS AL MUNDO DE LOS PROBLEMAS CLÁSICOS DE SINCRONIZACIÓN. AQUÍ, CADA PROCESO TIENE UN ROL, PERO NO SIEMPRE TRABAJAN EN ARMONÍA."



"¡YO TAMBIÉN QUIERO COMER, PERO SI TODOS TOMAMOS UN TENEDOR AL MISMO TIEMPO, QUEDAREMOS BLOQUEADOS!"

TENGO HAMBRE, PERO NECESITO DOS TENEDORES PARA COMER ESTE RESBALADIZO ESPAGUETI!"



"¡ESTAMOS ATRAPADOS! ESTO ES UN BLOQUEO MUTUO."



"USAREMOS SEMÁFOROS PARA EVITAR EL BLOQUEO. ADEMÁS, ESTABLECEREMOS REGLAS PARA QUE DOS FILÓSOFOS PUEDAN COMER SIMULTÁNEAMENTE SI SUS VECINOS NO ESTÁN COMIENDO."



"¡CON ESTA SOLUCIÓN, YA NO PASAREMOS HAMBRE NI NOS BLOQUEAREMOS!"

"¡Y PODEMOS PENSAR MIENTRAS ESPERAMOS NUESTRO TURNO!"



SINCRONIZANDO PROBLEMAS DE IPC

CAPITULO 2 "EL PROBLEMA DE LOS LECTORES Y ESCRITORES"

"ESTAR PREOCUPADO POR PROBLEMAS Y LEER MIENTRAS ESCRIBES"



"¡QUIERO LEER LOS DATOS, PERO EL ESCRITOR ME BLOQUEA!"

"¡NECESITO EXCLUSIVIDAD PARA ACTUALIZAR LA BASE DE DATOS!"



"¡ESTO ES INJUSTO! ¡NUNCA PUEDO ESCRIBIR PORQUE SIEMPRE HAY LECTORES ACTIVOS!"



"INTRODUCIREMOS UN SEMÁFORO Y UN CONTADOR PARA QUE MÚLTIPLES LECTORES PUEDAN LEER AL MISMO TIEMPO, PERO LOS ESCRITORES TENDRÁN PRIORIDAD CUANDO SEA NECESARIO."



"¡Y CUANDO ES MI TURNO, PUEDO ESCRIBIR SIN PROBLEMAS!"



"AHORA PODEMOS LEER JUNTOS SIN INTERRUPCIONES."



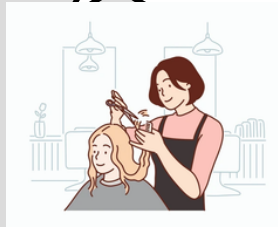
LA HISTORIA CONTINUA

SINCRONIZANDO PROBLEMAS DE IPC

CAPITULO 3 "EL PROBLEMA DEL PELUQUERO DORMIDO"

"CORTAR EL PELO Y PAGAR POR ELLO."

CLIENTE 1: "¿TENGO QUE DESPERTAR AL PELUQUERO PARA QUE ME ATIENDA?"

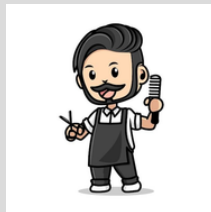


"SI EL PELUQUERO ESTÁ OCUPADO Y NO HAY SILLAS DISPONIBLES, LOS CLIENTES TENDRÁN QUE IRSE."

"USAREMOS SEMÁFOROS Y UN CONTADOR PARA GESTIONAR A LOS CLIENTES. LOS QUE ESPERAN TENDRÁN UNA SILLA ASIGNADA Y EL PELUQUERO SABRÁ A QUIÉN ATENDER."



"CON ESTE SISTEMA, PUEDO DORMIR CUANDO NO HAY CLIENTES Y ATENDERLOS EN ORDEN CUANDO LLEGAN."



"¡ESTO FUNCIONA PERFECTO! AHORA SÉ QUE SERÉ ATENDIDO SIN PROBLEMAS."



"GRACIAS A LA SINCRONIZACIÓN, LOS PROCESOS TRABAJAN EN ARMONÍA."

"RECUERDEN, LA CLAVE PARA UN SISTEMA EFICIENTE ES EVITAR BLOQUEOS, INANICIÓN Y CONFLICTOS."